

AGENTIC AI 시스템 아키텍처

내용

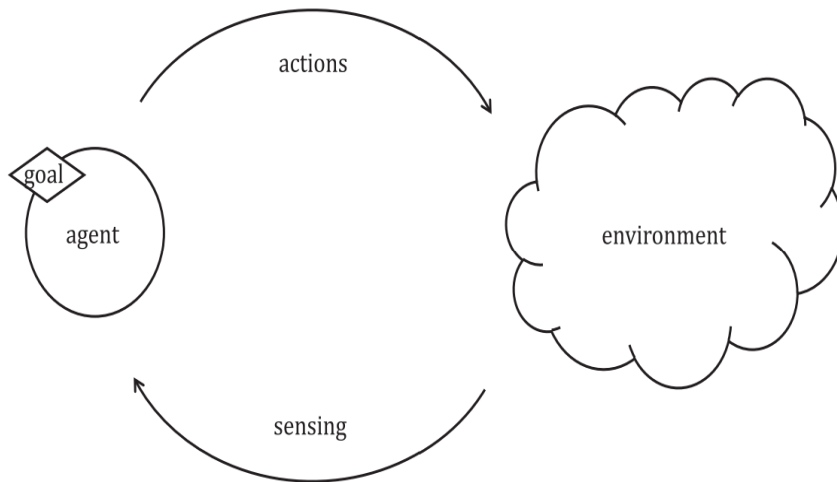
- ❖ Agent 시스템 개요
- ❖ Agent 핵심 기술 요소
- ❖ Agentic AI 논리적 아키텍처
- ❖ 사례 연구

AGENTIC AI 시스템 개요

Agent 개념

Agent 란?

특정 목표(Goal)를 달성하기 위해 주변 환경(Environment)을 인식하고(Sensing), 그 결과를 바탕으로 행동(Action)을 수행하는 자율적 소프트웨어 또는 시스템



Applications of the Multi-agent system

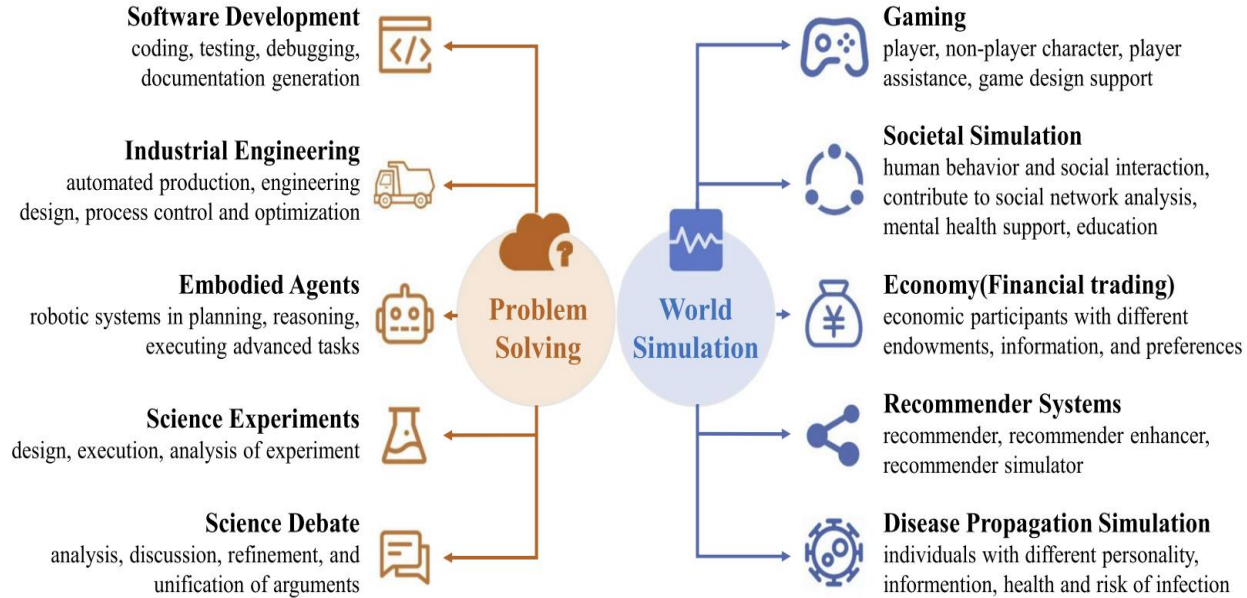
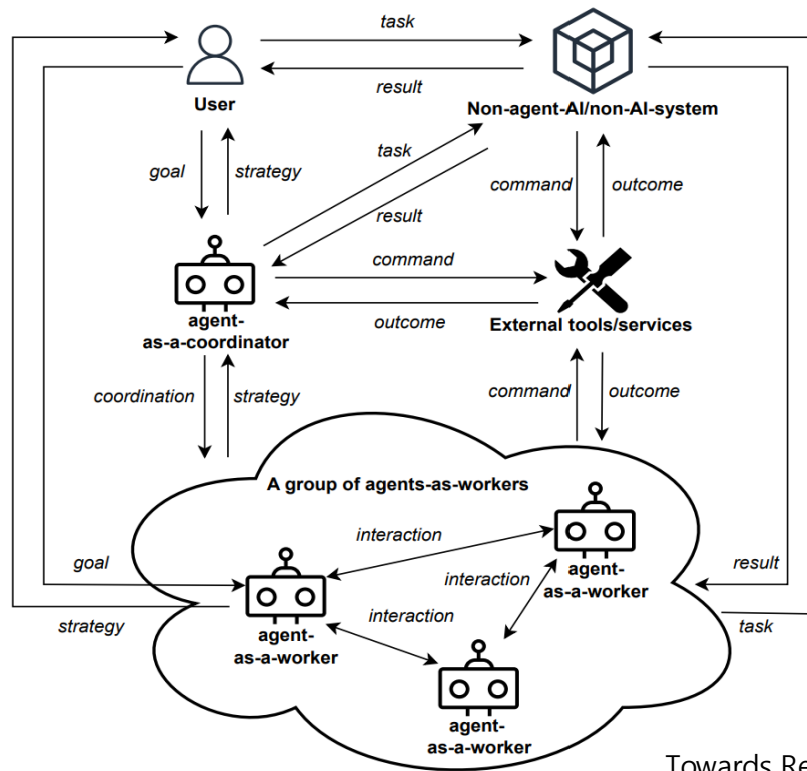


Fig. 7 Diverse application of the Llm-based multi-agent system

Agent 핵심 요소

핵심 요소	주요 역할	설명	기여
Planning (계획)	과업 분해 및 로드맵 설계	복잡하고 추상적인 목표를 논리적 순서에 따른 하위 작업(Sub-tasks)으로 구조화함.	목적 지향성: 중장기 목표를 잃지 않고 체계적으로 접근함.
Memory (기억)	Context 유지 및 지식 축적	단기 기억: 현재 워크플로우 내 대화/로그 저장 장기 기억: 과거 경험과 외부 지식을 벡터 DB 등에 저장/참조.	일관성: 과거의 맥락을 이해하고 학습된 지식을 적시에 활용함.
Tool Use (도구 활용)	물리적/디지털 실행력	텍스트 생성을 넘어 웹 브라우저, API 호출, 데이터베이스 조회, 코드 실행 등 외부 환경과 상호작용함.	확장성: 지능을 실제 행동으로 전환하여 실질적인 과업을 완결함.
Self-Reflection (자기 성찰)	품질 통제 및 자가 교정	자신의 추론 과정이나 도구 실행 결과에 오류가 있는지 비판적으로 검토하고 개선안을 도출함.	신뢰성: 실패 시 포기하지 않고 대안을 찾아 성공 확률을 극대화함.

Agentic AI 시스템의 개념도



Towards Responsible Generative AI:
A Reference Architecture For Designing Foundation Model Based Agents, IEEE Software 2024

Agentic AI 시스템 아키텍처

❖ Data Flow View

사용자 요청



Context 생성 (Context Engineering)



Prompt 생성 (Prompt Engineering)



기억 검색 (Memory)



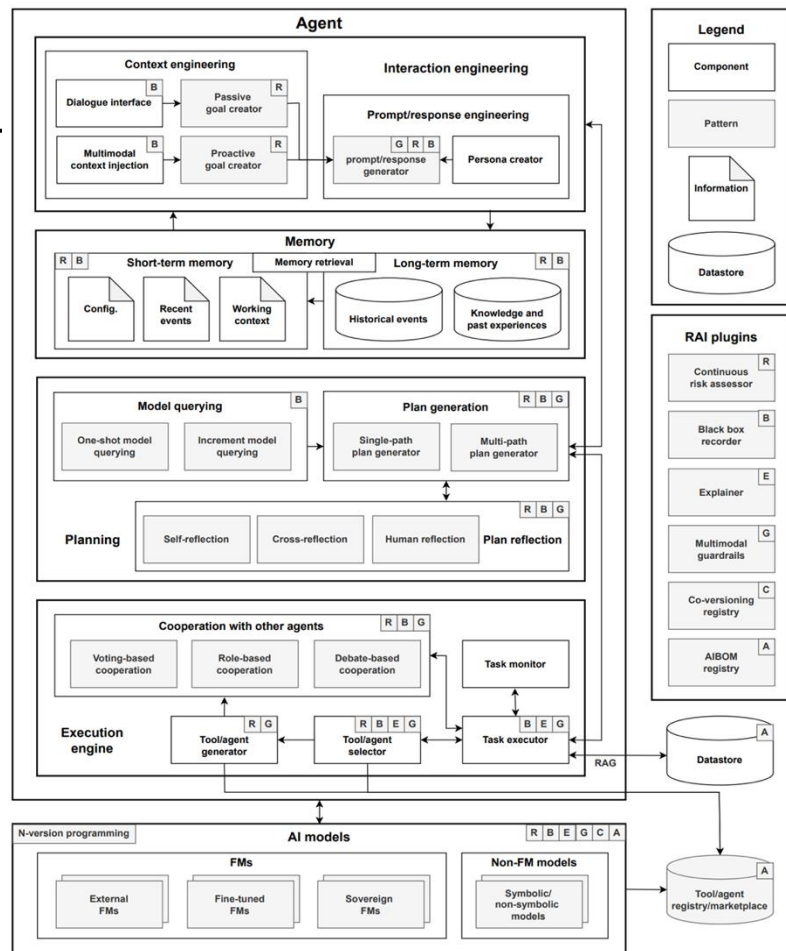
계획 수립 (Planning)



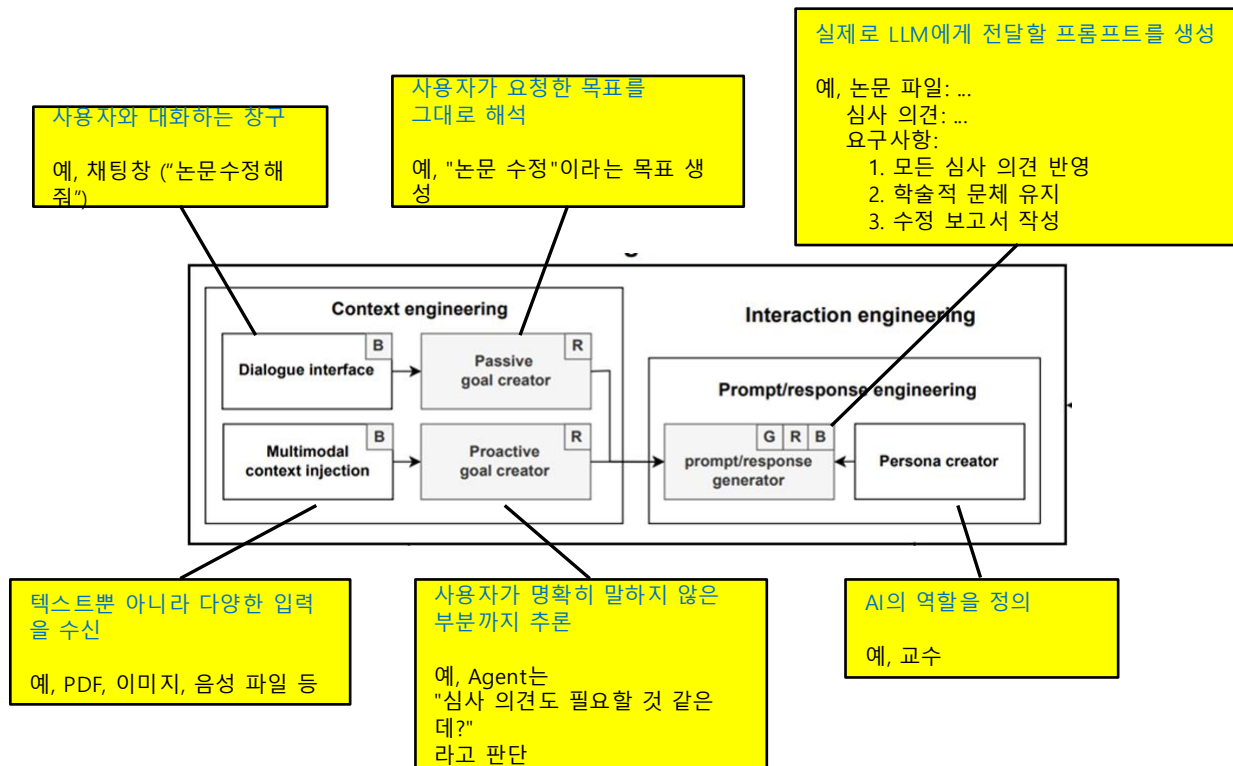
작업 실행 (Execution Engine)



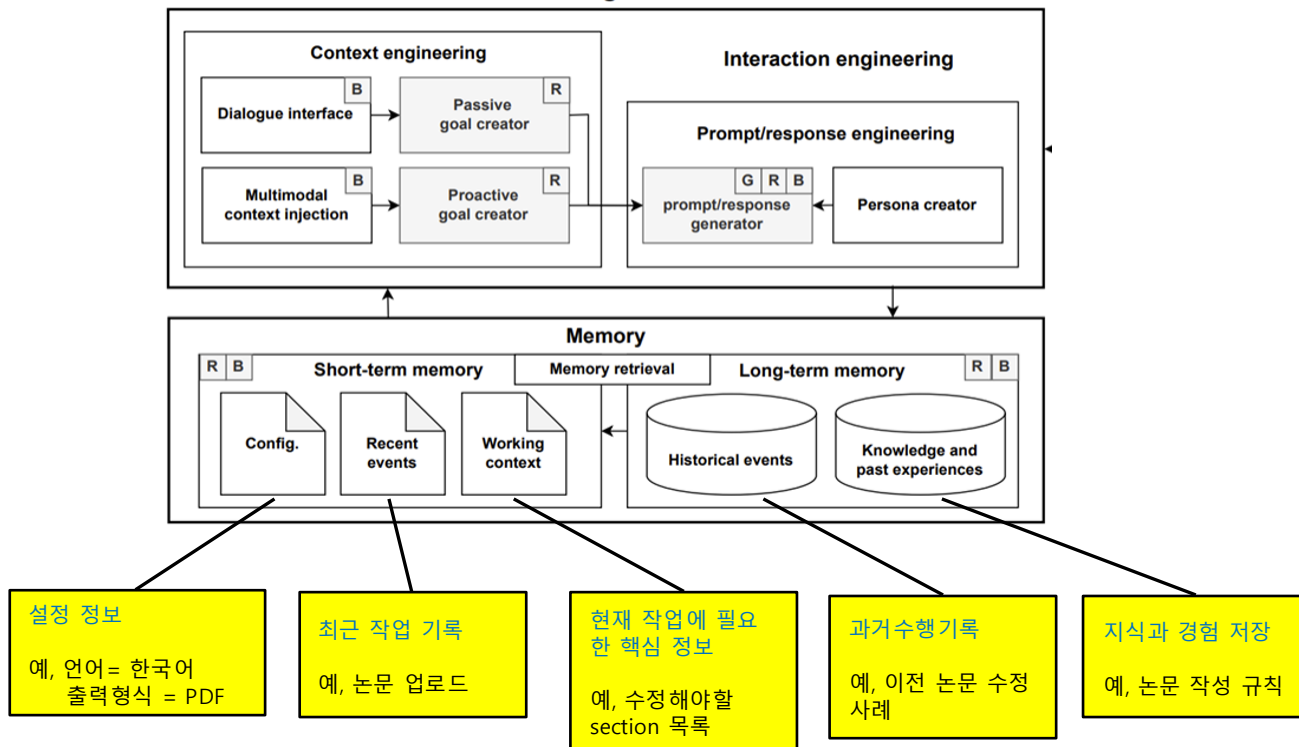
결과 생성



Interaction Engineering



Memory



Planning

LLM에게 질문하는 과정

예, “심사의견을 어떻게 처리할까?”를 모델에게 물어본다.

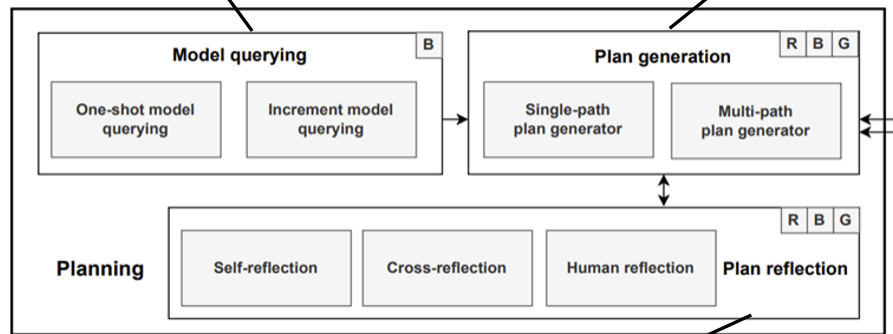
One-shot Query: 한번에 질문

Increment Query: 여러번 나누어 점진적인 질문

실행 계획 생성

Single-path plan generator: 단일 경로 계획 생성

Multi-path plan generator: 다중 경로 계획 생성 및 비교



Agent의 자기 검토

Self-reflection: 자기검토 기능

Cross-reflection: 다른 Agent의 결과 검토

Human-reflection: 사용자 피드백 반영

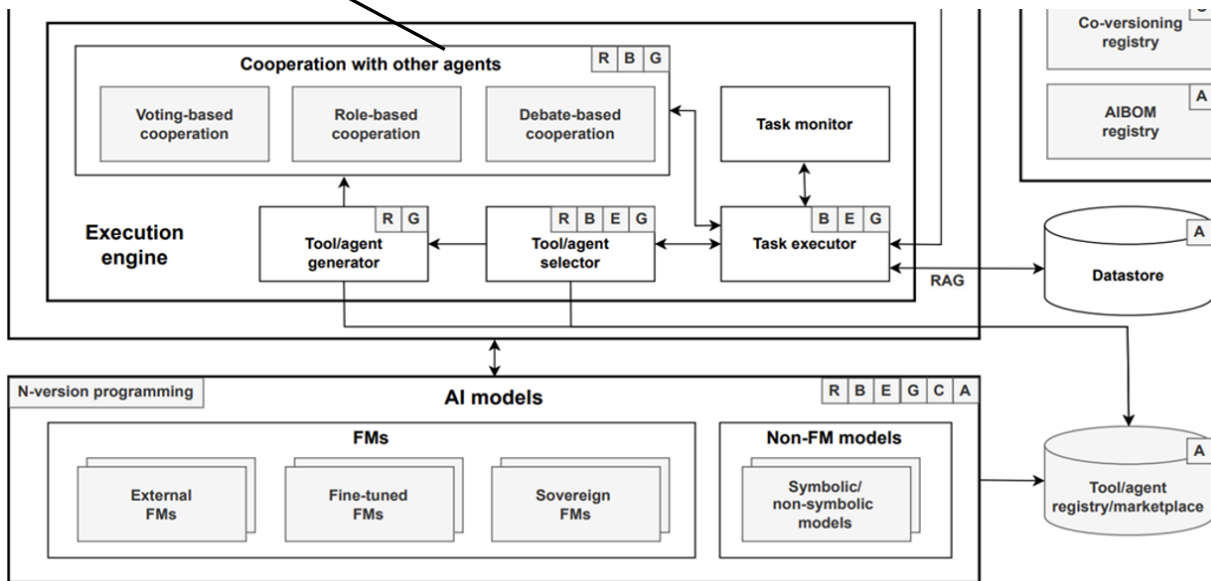
Execution Engine

여러 Agent간의 협업

Voting-based cooperation: 여러 Agent가 투표하여 결정

Role-based cooperation: Agent에 역할을 분담하여 협업

Debate-based cooperation: Agent들이 토론하면서 조정

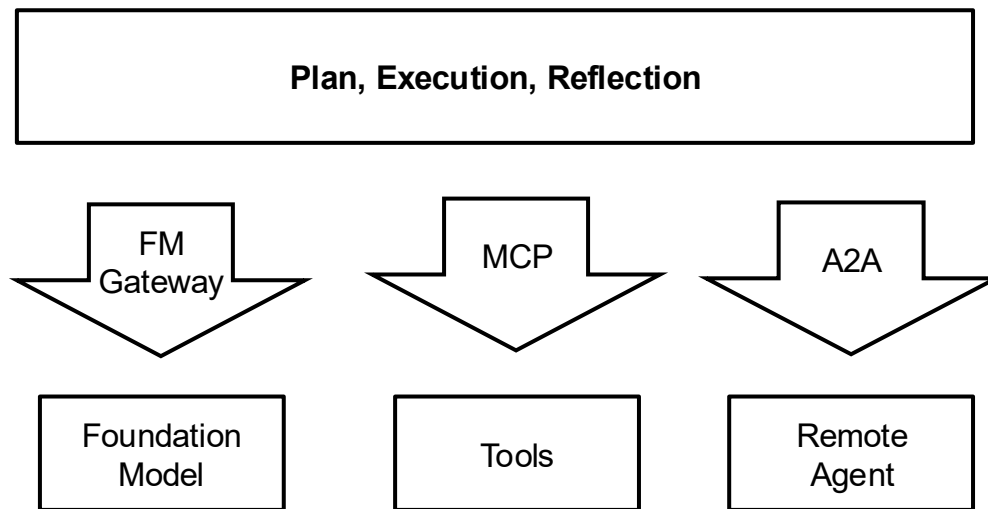


LLM(Classic Chat) vs Agentic AI

구분 항목	기존 LLM (Classic Chat)	Agentic AI
과업 지향점	Prompt-centric (지시어 수행)	Goal-oriented (목표 달성)
제어 흐름	선형적 (Linear) Input → Output	순환적 (Iterative Loop) Plan → Act → Observe → Reflect
Planning	정적 나열: 계획을 글로 씀	동적 오케스트레이션: 결과에 따라 계획 수정
Memory	Sliding Window: 단순 컨텍스트 요약	Cognitive Memory: 장/단기 기억의 구조화
Self-Correction	외부 루프: 인간이 지적해야 수정	내부 루프: 사용자 노출 전 자가 검토
Tool Use	기능 호출: 개발자가 정의한 대로 실행	환경 상호작용: 에이전트가 도구 선택
실패 대응	에러 출력 후 사용자의 지시 대기	에러 분석 후 자율적으로 재시도/우회

AGENT 핵심 기술 요소

Core Concepts

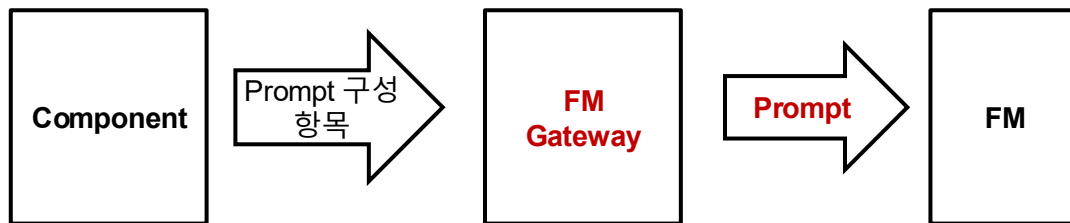


FM Gateway, MCP, and A2A

연결 대상	핵심 역할	연결 방식
Foundation Model	이해, 추론, 계획, 생성	FM Gateway
Tool / Resource / Prompt	외부 도구와 데이터 접근	MCP
Remote Agent	Agent 간 협업과 task 위임	A2A

FM Gateway

- ❖ Prompt = Instruction + Task Description + Model Context + Constraints+ Output Schema + Examples + Policy Instruction



- ❖ FM Gateway

책임	설명
Model Routing	요청 목적, 정확성, 비용, latency, 보안 기준으로 FM 선택
Prompt Assembly	instruction, context, schema, policy를 통합해 prompt 구성
Output Schema Enforcement	JSON schema 등 구조화된 출력 요구
Response Parsing	Raw model response를 시스템이 읽기 쉬운 구조화된 데이터로 변환

Model Context Protocol

❖ 개념

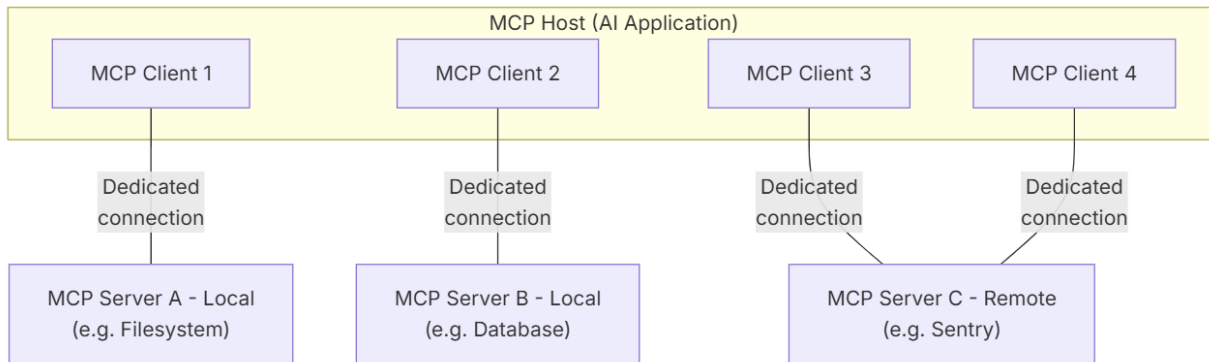
- Agent가 외부 tool, resource, workflow를 사용할 수 있게 하는 실행 인터페이스

❖ Primitives

- What clients and servers can offer each other

유형	MCP Server가 제공하는 것	예	Agent 시스템에서의 의미
Tool	실행 가능한 함수	file operations, API calls, database queries	외부 기능 실행
Resource	읽기 가능한 data/context	file contents, database records, API responses	Model Context 구성
Prompt	재사용 가능한 prompt template	system prompts, few-shot examples	일관된 FM 작업 절차 재사용

MCP Host/Client and Server



구분	Stdio Transport	Streamable HTTP Transport
기본 개념	표준 IO스트림을 이용해 로컬 프로세스 간 직접 통신	HTTP 기반으로 원격 MCP Server와 통신
통신 방식	stdin / stdout	HTTP POST + 선택적 Server-Sent Events(SSE)
적합한 경우	로컬 파일 접근, 로컬 DB 조회, 로컬 도구 실행, 개발 환경, 폐쇄망 환경	외부 API, SaaS, 클라우드 서비스, 원격 도구 서버, 조직 간 MCP 연동
장점	빠름, 단순함, 네트워크 설정 불필요, 로컬 자원 접근 용이	원격 접근 가능, 표준 HTTP 인프라 활용, 인증/인가 체계 적용 용이
단점	같은 머신 또는 로컬 실행 환경에 제한	네트워크 지연, 인증, 보안, 장애 대응 필요

Tool: MCP 서버 예

```
# MCP Server: paper_tools_server.py
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("paper-tools-server")

@mcp.tool()
def extract_section(paper_id: str, section_name: str) -> dict:
    # Extract a section from a paper.
    return {
        "paper_id": paper_id, "section_name": section_name,
        "text": "... extracted section text ..."
    }

@mcp.tool()
def extract_reviewer_comments(review_file_id: str) -> list[dict]:
    # Extract reviewer comments from a review file.
    return [ {
        "comment_id": "C1",
        "text": "The methodology section lacks sufficient detail." }
    ]

if __name__ == "__main__":
    mcp.run()
```

Tool: MCP 클라이언트 예

```
# MCP Host / Client 측 pseudo code

client = connect_to_mcp_server("paper-tools-server")

# 1. Tool 목록 조회
tools = client.list_tools()

# 2. 필요한 Tool 선택
# 예: reviewer_comments를 추출해야 함
result = client.call_tool(
    name="extract_reviewer_comments",
    arguments={
        "review_file_id": "artifact://review_comments.pdf"
    }
)

# 3. 결과를 Task Context 또는 Inter-Task Workspace에 반영
task_workspace["reviewer_comments"] = result
```

Resource: MCP 서버 예

```
# MCP Server: paper_resource_server.py
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("paper-resource-server")

@mcp.resource("paper://{paper_id}/metadata")
def paper_metadata(paper_id: str) -> str:
    # Return paper metadata.
    return """
    Title: An Architecture for AI Agent Systems
    Sections: Abstract, Introduction, Method, Experiment, Conclusion
    Status: Submitted version
    """

@mcp.resource("review://{review_id}/summary")
def review_summary(review_id: str) -> str:
    #Return reviewer comment summary.
    return """
    Reviewer comments include:
    - Need more details in methodology
    - Need stronger comparison with baselines
    - Clarify contribution
    """

if __name__ == "__main__":
    mcp.run()
```

Resource: MCP 클라이언트 예

```
# MCP Host / Client 측 pseudo code

client = connect_to_mcp_server("paper-resource-server")

# 1. Resource 목록 조회
resources = client.list_resources()

# 2. 특정 Resource 읽기
metadata = client.read_resource("paper://paper_v3/metadata")
review_summary = client.read_resource("review://review_001/summary")

# 3. 읽은 Resource를 Model Context에 포함
model_context = {
    "paper_metadata": metadata,
    "review_summary": review_summary
}
```

Prompt: MCP 서버 예

```
# MCP Server: paper_prompt_server.py
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("paper-prompt-server")

@mcp.prompt()
def draft_response_letter(
    reviewer_comment: str, revision_summary: str, tone: str = "polite and academic"
) -> str:
    # Return a reusable prompt template for drafting a response letter.
    return f"""
    You are an academic writing assistant.
    Draft a response to the reviewer comment.
    Reviewer Comment:
    {reviewer_comment}
    Revision Summary:
    {revision_summary}
    Tone:
    {tone}
    Requirements:
    - Be polite and professional.
    - Explain how the manuscript was revised.
    - Do not invent unsupported changes.
    """

if __name__ == "__main__":
    mcp.run()
```


Prompt: MCP 클라이언트 예

```
# MCP Host / Client 측 pseudo code
client = connect_to_mcp_server("paper-prompt-server")

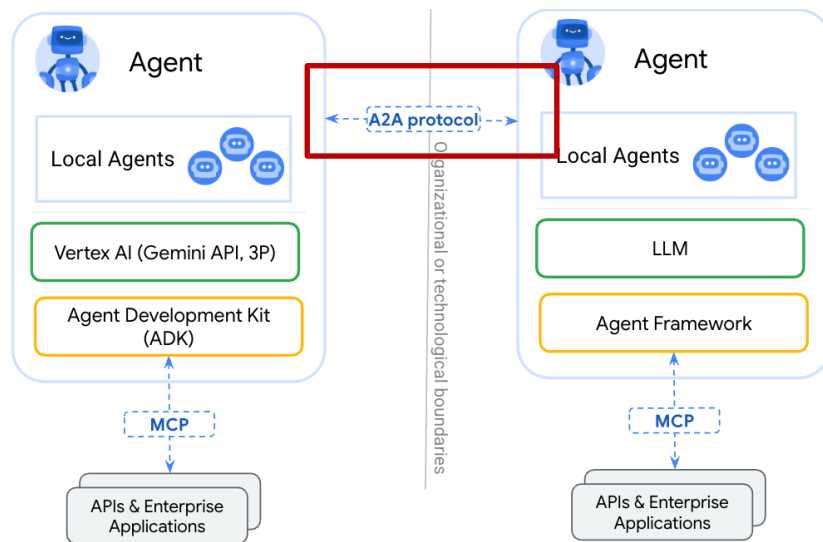
# 1. Prompt 목록 조회
prompts = client.list_prompts()

# 2. Prompt template 가져오기
prompt = client.get_prompt(
    name="draft_response_letter",
    arguments={
        "reviewer_comment": "The methodology section lacks sufficient detail.",
        "revision_summary": "Section 3.2 was expanded with algorithm details.",
        "tone": "polite and academic"
    }
)

# 3. FM Gateway가 최종 Prompt 구성에 활용
fm_response = fm_gateway.invoke(
    prompt=prompt,
    output_schema="CommentResponse"
)
```

A2A (Agent2Agent) Protocol

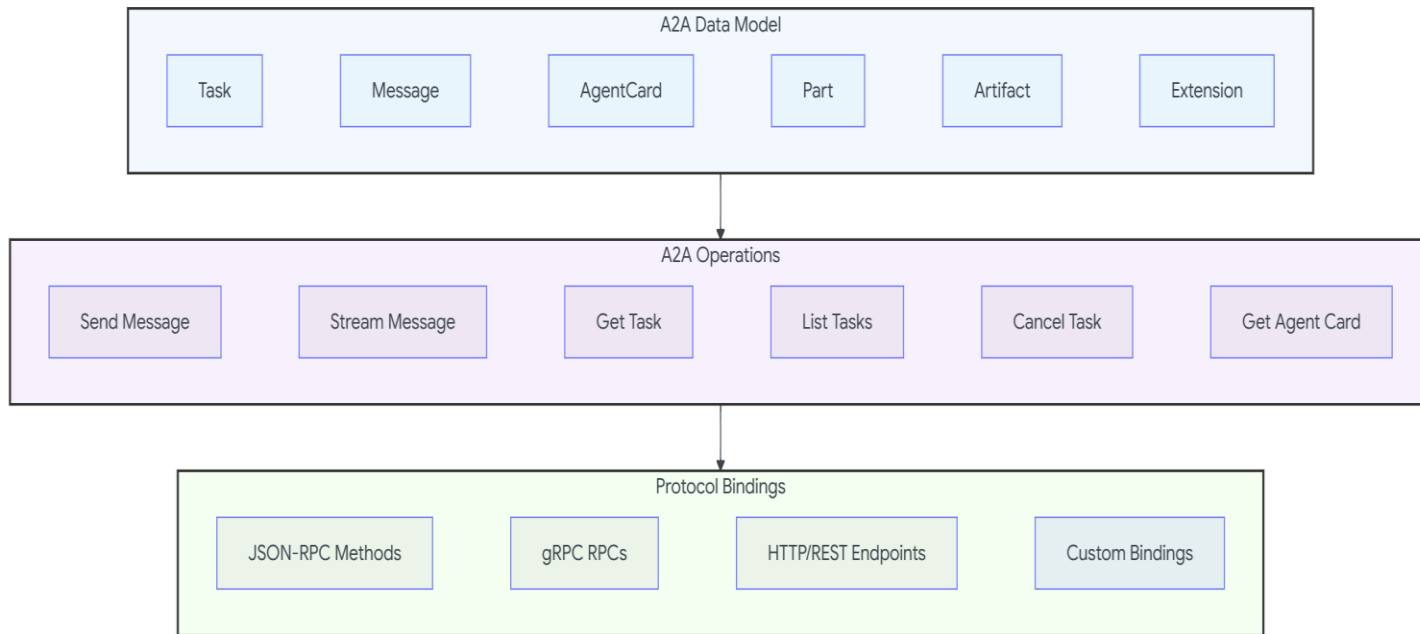
- ❖ The A2A protocol is an open standard that enables seamless **communication and collaboration between AI agents**
- ❖ A2A allows agents from different developers, built on different frameworks, and owned by different organizations to unite and work together.



A2A 역할

역할	설명
Remote Agent Discovery	외부 Agent의 capability 확인
Agent Delegation	특정 Subgoal 또는 Task를 Remote Agent에게 위임
Task Management	long-running task 상태, 완료, 실패, 취소 관리
Context Exchange	필요한 context, instruction, artifact 전달
Artifact Exchange	Remote Agent가 생성한 결과물 전달
Agent Interoperability	다른 framework/vendor/language로 만든 Agent와 협업

A2A Protocol



A2A Data Model

- ❖ Core data structures and message formats that all A2A implementations must understand

요소	설명
Agent Card	Agent의 capability, endpoint, protocol, security 정보를 광고하는 metadata
Message	Agent 간 context, instruction, reply, artifact 등을 주고받는 통신 단위
Task	Remote Agent에게 위임된 수행 단위
Artifact	task 수행 결과물
Parts	message/artifact 내부의 typed content unit

A2A Operations

❖ Fundamental capabilities and behaviors that A2A agents must support,

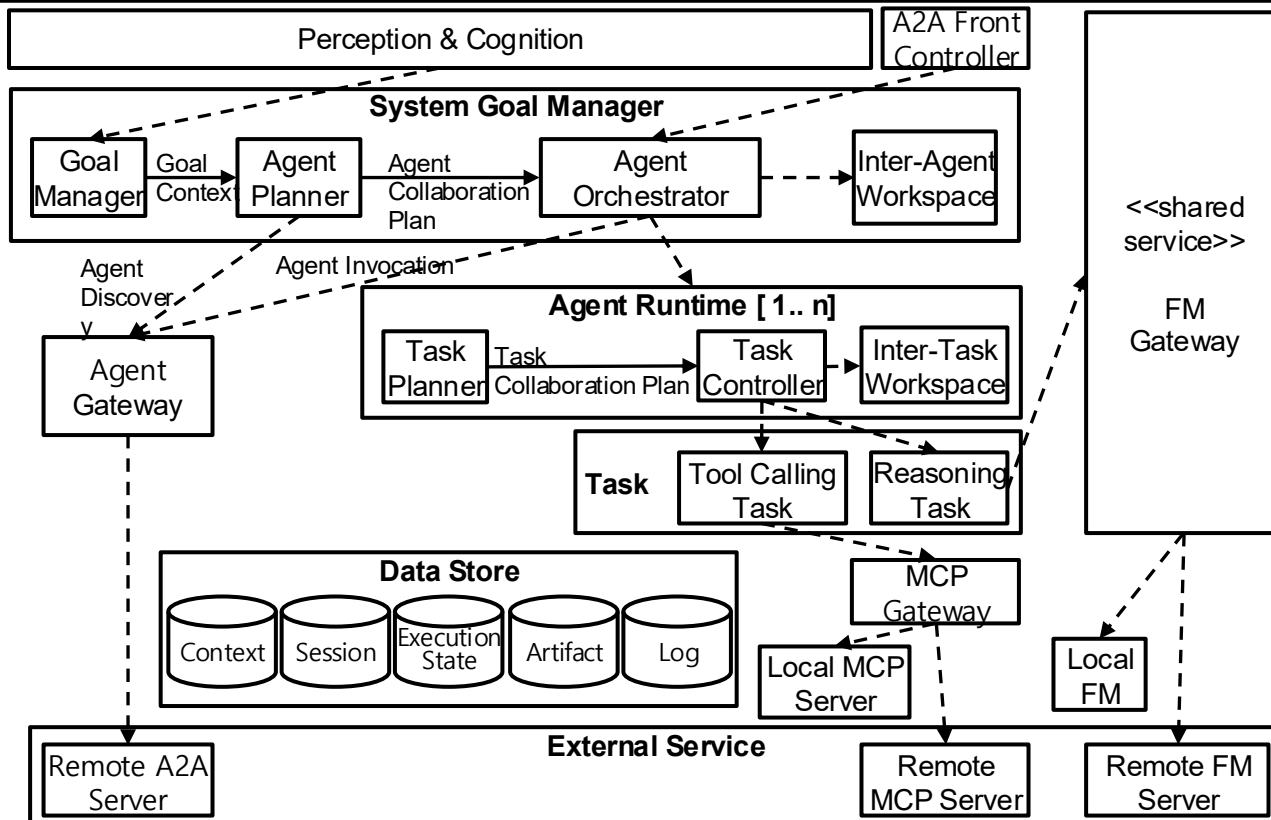
A2A Operation	설명
Send Message	원격 Agent에게 메시지를 보내 작업을 요청하거나 기존 작업을 이어서 수행하게 함
Stream Message	메시지를 보내고, 원격 Agent의 진행 상황과 결과를 스트리밍으로 수신
Get Task	특정 Task의 현재 상태와 결과를 조회
List Tasks	특정 조건에 해당하는 Task 목록을 조회
Cancel Task	진행 중인 Task의 취소를 요청
Get Agent Card	원격 Agent의 capability, endpoint, protocol, 인증 정보 등을 조회

MCP vs A2A

구분	MCP (Model Context Protocol)	A2A (Agent-to-Agent)
핵심 개념	모델과 외부 데이터/도구 간의 표준 인터페이스	독립적인 AI 에이전트 간의 상호 협력 및 통신
주요 목적	도구 재사용성 극대화 및 데이터 통합 비용 절감	복잡한 태스크를 분업화하여 지능적 해결
통신 대상	에이전트 ↔ 데이터/기능(Server)	에이전트 A ↔ 에이전트 B
비유	"비서가 쓸 수 있는 범용 도구 세트"	"팀장 비서와 실행 비서 간의 업무 대화"
데이터 흐름	정형화된 도구 호출 및 리소스 반환	자연어 기반의 지시, 질문, 결과 보고

논리적 아키텍처

Agentic AI 시스템 논리적 아키텍처



Goal Manager

❖ 핵심 개념

- Goal Manager는 Request / Session Context를 바탕으로 시스템이 달성해야 할 목표를 명확한 Goal Context로 정의
- 사용자의 자연어 요청을 그대로 실행하지 않고, 수행 범위, 입력, 기대 산출물, 성공 기준, 제약 조건을 명시한 목표 단위로 변환

❖ 주요 책임

구분	설명
Goal 식별	사용자 요청에서 시스템 목표 도출
Goal Context 생성	goal statement, inputs, outputs, success criteria 구성
성공 기준 정의	결과가 성공적인지 판단할 기준 생성
제약 조건 정리	품질, 정책, 도메인 제약 식별
누락 정보 식별	goal 수행 전 필요한 추가 정보 확인
Goal 검증	요청 의도, artifact, 정책과 goal의 일관성 확인

Agent Planner

❖ 핵심 개념

- Goal Context를 바탕으로 목표를 여러 Subgoal로 분해하고, 각 Subgoal을 어떤 Agent가 수행할지 결정하여 Agent Collaboration Plan을 생성
- 즉, 시스템 수준 목표를 Agent 협업 구조로 변환하는 planning 컴포넌트

❖ 주요 책임

구분	설명
Goal 분해	Goal을 Subgoal Graph로 분해
Agent Discovery	Agent Gateway를 통해 사용 가능한 Agent 탐색
Capability Matching	Subgoal 요구 능력과 Agent capability 매칭
Agent 할당	각 Subgoal에 Local 또는 Remote Agent 할당
협업 구조 결정	Agent 간 control/data dependency 정의
Agent Collaboration Plan 생성	Agent Orchestrator가 실행할 계획 산출

Agent Orchestrator

❖ 핵심 개념

- Agent Planner가 생성한 Agent Collaboration Plan을 실행
- Local Agent Runtime을 직접 실행하고, Remote Agent는 Agent Gateway를 통해 A2A 방식으로 호출하고, Agent 결과를 수집, 조정, 통합

❖ 주요 책임

구분	설명
Plan 실행	Agent Collaboration Plan 실행
Local Agent 실행	Agent Runtime [1..n] 호출
Remote Agent 호출	Agent Gateway를 통해 원격 Agent 호출
협업 상태 관리	Agent 실행 상태, Agent dependency, 실패/재시도 관리
결과 통합	여러 Agent 결과를 Goal 결과로 통합

Task Planner

❖ 핵심 개념

- 특정 Agent Runtime에 할당된 Subgoal을 실행 가능한 Task Graph로 분해하고, Task Collaboration Plan을 생성
- 즉, Agent 내부에서 Subgoal을 Task 수준 실행 구조로 변환

❖ 주요 책임

구분	설명
Subgoal 분석	Agent Runtime Context를 바탕으로 수행 목표 이해
Task 분해	Subgoal을 여러 Task로 분해
Task 유형 결정	Reasoning Task, Tool Calling Task 등으로 분류
Task 의존성 정의	Task 간 control/data dependency 정의
Task Context 생성	각 Task 수행에 필요한 맥락 정의
Task Collaboration Plan 생성	Task Controller가 실행할 계획 산출

Task Controller

❖ 핵심 개념

- Task Planner가 생성한 Task Collaboration Plan을 실행
- Task 실행 순서, dependency, retry, failure handling, context 전달, workspace update를 관리

❖ 주요 책임

구분	설명
Task 실행 제어	Task Graph에 따라 Task 실행
dependency 관리	선행 Task 완료 여부와 입력 준비 상태 확인
Task Context 제공	각 Task에 필요한 context 구성 및 전달
Reasoning Task 호출	FM Gateway 기반 reasoning/generation 실행
Tool Calling Task 호출	MCP Gateway 기반 tool 실행
실패 처리	retry, fallback, skip, error propagation 수행
실행 상태 기록	Task 상태를 Execution State Store에 기록

Task

❖ 핵심 개념

- Agent Runtime 내부에서 실제 작업이 수행되는 실행 단위
- Task를 크게 Reasoning Task와 Tool Calling Task로 구분

❖ 주요 책임

구분	설명
작업 실행	Task Controller의 지시에 따라 실제 작업 수행
context 사용	Task Context를 이용해 수행 목적과 제약 이해
결과 생성	후속 Task나 Agent Runtime이 사용할 결과 생성
workspace update	Task 결과를 Inter-Task Workspace에 반영
로그 기록	주요 실행 결과와 오류를 Log Store에 기록

Reasoning Task

❖ 핵심 개념

- 외부 Tool 호출 없이 FM Gateway를 통해 판단, 분류, 요약, 생성, 검증 등의 지능형 처리를 수행
- 전통적 함수 호출이 아니라, Task Context를 기반으로 FM을 호출하여 구조화된 결과를 생성

❖ 주요 책임

구분	설명
Reasoning 수행	판단, 추론, 분류, 요약, 생성 수행
Model Context 구성 요청	Task Context 중 FM에 필요한 부분 선택
FM 호출	FM Gateway를 통해 Local/Remote FM 사용
Structured Output 생성	Schema에 맞는 결과 반환
결과 검증	출력 형식, 제약 준수, 일관성 확인

Tool Calling Task

❖ 핵심 개념

- 외부 도구, API, DB, 파일 처리, 계산, 실행 환경 등을 사용
- MCP Gateway를 통해 Local 또는 Remote MCP Server의 Tool/Resource를 호출

❖ 주요 책임

구분	설명
Tool 필요성 판단	Task 수행에 외부 도구가 필요한지 판단
Tool Context 구성	tool 호출 목적, 대상 resource, 기대 결과 정의
Tool 선택	MCP Gateway를 통해 적절한 tool 선택
Argument 구성	Task Context를 tool argument로 변환
Tool 결과 해석	raw result를 task output으로 정규화

FM Gateway

❖ 핵심 개념

- 모든 Foundation Model 접근을 중재하는 shared service
- 각 컴포넌트가 직접 Local/Remote FM을 호출하지 않고, FM Gateway를 통해 FM 선택, Prompt 구성, Structured Output Parsing, Logging을 수행

❖ 주요 책임

구분	설명
모델 라우팅	요청 목적, 정확성, 비용, latency 기준으로 모델 선택
Model Context 구성	FM 호출에 포함할 context subset 선택
Prompt 생성	instruction, context, schema, policy를 prompt로 조립
FM 호출	Local FM 또는 Remote FM Server 호출
응답 정규화	raw model response를 structured output으로 변환
로그 기록	prompt, response, token, cost, latency 기록

Local FM vs Remote FM Server

구분	Local FM	Remote FM Server
핵심 개념	시스템 내부 또는 동일 배포 환경에서 직접 실행되는 Foundation Model	외부 클라우드/API/원격 서버를 통해 호출되는 Foundation Model
위치	로컬 서버, 온프레미스, 엣지 장비, 내부 GPU 서버	외부 FM 제공자, 클라우드 모델 API, 원격 사내 모델 서버
호출 방식	FM Gateway가 내부 endpoint 또는 local runtime 호출	FM Gateway가 네트워크 API를 통해 원격 모델 호출
주요 장점	데이터 외부 유출 위험이 낮고, 네트워크 지연이 적으며, 운영 통제가 쉬움	고성능·최신 모델을 쉽게 활용할 수 있고, 모델 운영 부담이 낮음
주요 단점	모델 크기, 추론 성능, GPU/메모리 자원에 제약이 있음	비용, latency, rate limit, 데이터 보안, 외부 의존성 문제가 있음
적합한 상황	민감 데이터 처리, 내부 문서 분석, 낮은 latency 요구, 오프라인/폐쇄망 환경	고난도 추론, 대규모 생성, 최신 모델 성능 필요, 내부 자원이 부족한 경우

MCP Gateway

❖ 핵심 개념

- Tool Calling Task가 MCP Server의 tool/resource/prompt 기능을 사용할 수 있도록 중재
- Task가 MCP 프로토콜에 의존하지 않도록 tool discovery, argument binding, invocation, result normalization을 담당

❖ 주요 책임

구분	설명
MCP Server 관리	Local/Remote MCP Server 연결 관리
Tool discovery	사용 가능한 tool/resource/prompt 조회
Tool selection 지원	Tool Context에 적합한 tool 선택
Argument binding	Task Context를 tool argument로 변환
Tool invocation	MCP Server에 tool 호출 요청
Result normalization	raw result를 task output 형식으로 변환
Error handling	timeout, retry, fallback 처리
로그 기록	MCP/tool 호출 기록 저장

Local MCP Server vs Remote MCP Server

구분	Local MCP Server	Remote MCP Server
핵심 개념	에이전트 시스템 내부 또는 동일 실행 환경에 배치된 MCP Server	네트워크를 통해 접근하는 외부 MCP Server
위치	로컬 PC, 내부 서버, 컨테이너, 같은 VPC/클러스터	외부 서버, 클라우드 서비스, 타 조직 시스템, SaaS API gateway
호출 방식	MCP Gateway가 내부 endpoint 또는 local process를 호출	MCP Gateway가 네트워크를 통해 원격 MCP endpoint 호출
제공 기능	로컬 파일 처리, 내부 DB 조회, 코드 실행, 문서 파싱, 사내 API 접근	외부 검색, 외부 DB/API, SaaS 연동, 원격 분석 서비스, 타 조직 tool 사용
주요 장점	latency가 낮고, 보안 통제가 쉽고, 내부 자원 접근이 용이함	내부에 없는 기능을 확장할 수 있고, 외부 전문 tool/service를 재사용 가능
주요 단점	로컬 환경에 설치된 tool과 resource에 기능이 제한됨	네트워크 latency, 인증, 장애, 보안, 외부 의존성 문제가 발생
적합한 상황	내부 문서 처리, 민감 데이터 분석, 로컬 파일 접근, 폐쇄망 환경	외부 지식 검색, 클라우드 API 호출, 타 시스템 연동, 고성능 외부 tool 사용

Agent Gateway

❖ 핵심 개념

- 외부 Remote Agent를 탐색하고 호출하기 위한 컴포넌트
- Agent Planner는 Agent Discovery에 사용하고, Agent Orchestrator는 Remote Agent Invocation에 사용

❖ 주요 책임

구분	설명
Agent discovery	사용 가능한 Remote Agent 탐색
Capability query	Remote Agent의 capability, endpoint, contract 확인
Agent invocation	A2A 방식으로 Remote Agent 호출
Context packaging	Remote Agent에게 전달할 context subset 구성
Response normalization	Remote Agent 응답을 내부 result 형식으로 변환
Failure handling	timeout, unavailable, retry, fallback 처리
로그 기록	A2A 호출과 응답 기록

사례 연구

논문 수정과 Response Letter 작성 사례로
이해하는 Agentic AI 시스템

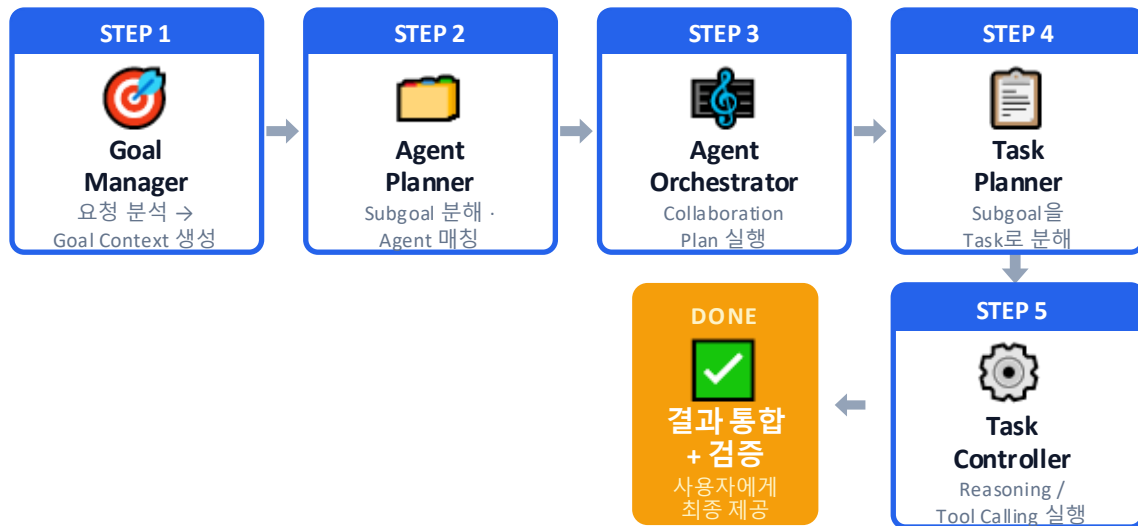
한 줄의 요청 → 6단계 협업 파이프라인

 User Request

“심사 의견을 반영하여 논문을 수정하고 Response Letter를 작성해 주세요.”

 인지 · 계획 (Cognition & Planning)

 오케스트레이션 (Orchestration)



 실행 (Execution)

STEP 1

Goal Manager (요청을 Goal Context로 번역)

사용자가 한 말

"심사 의견을 반영해서
논문을 수정하고 답변서 좀 써줘."

↓ Goal Manager가 분석 & 구조화

생성된 Goal Context

```
goal_context.json

"goal": "논문 수정 + Response Letter 작성"
"inputs": [ paper.pdf, reviews.txt ]
"outputs": [ revised_paper, response_letter ]
"constraints":
  ✓ 모든 심사 의견 반영
  ✓ 수정 내용과 논문의 일치
  ✓ 학술적 문체, no fabrication
```

Goal Manager가 하는 일

① 목표 식별

"수정"과 "답변서 작성"
두 개의 명확한 목표로 분리

② 성공 기준 정의

"어떻게 되어야 끝난 것인가?"
에 대한 측정 가능한 기준.

③ 제약 조건 정리

품질·정책 제약을 명시
(예: 데이터 위조 금지).

④ 누락 정보 확인

필요한 입력 파일이 모두 준비
되었는지 검사합니다.

💡 KEY MESSAGE

Goal Manager는 "인간의 언어"를
"기계가 실행할 수 있는 명세서"로 번역합니다.

STEP
2

Agent Planner

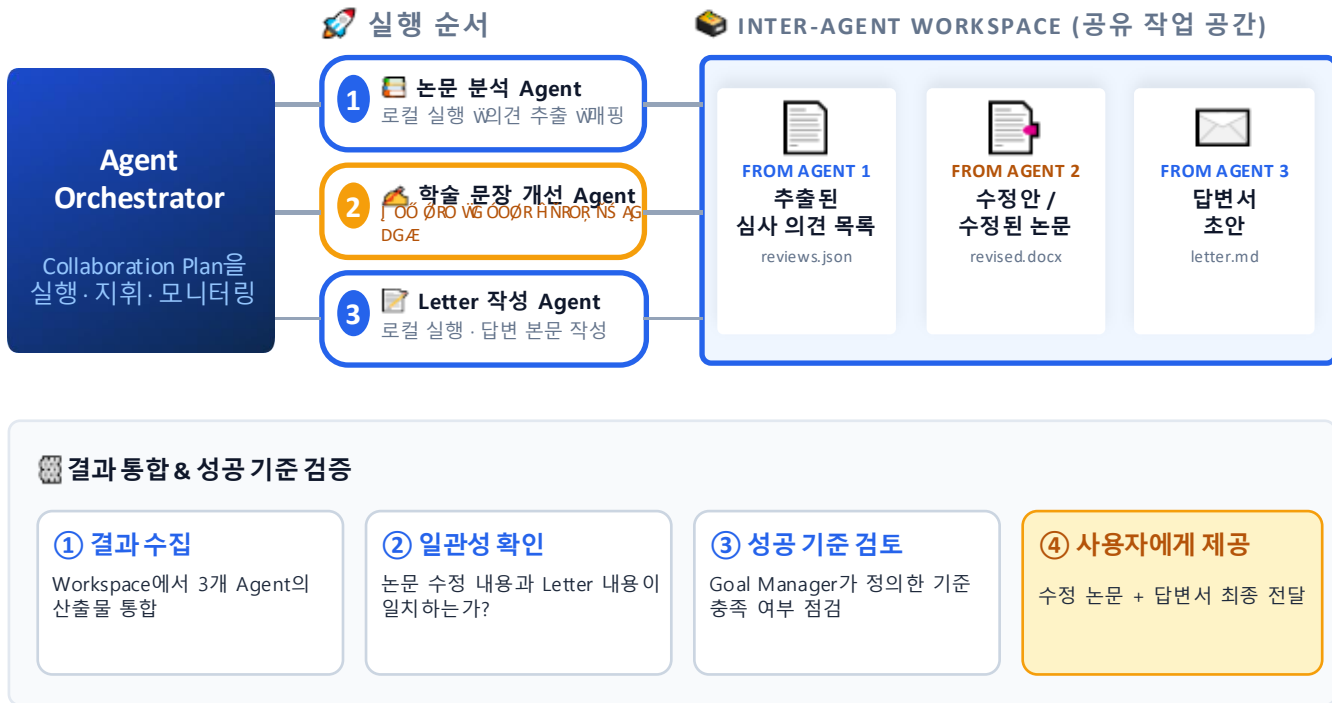
(목표를 Subgoal로 쪼개고, Agent에 분배)



STEP
3

Agent Orchestrator

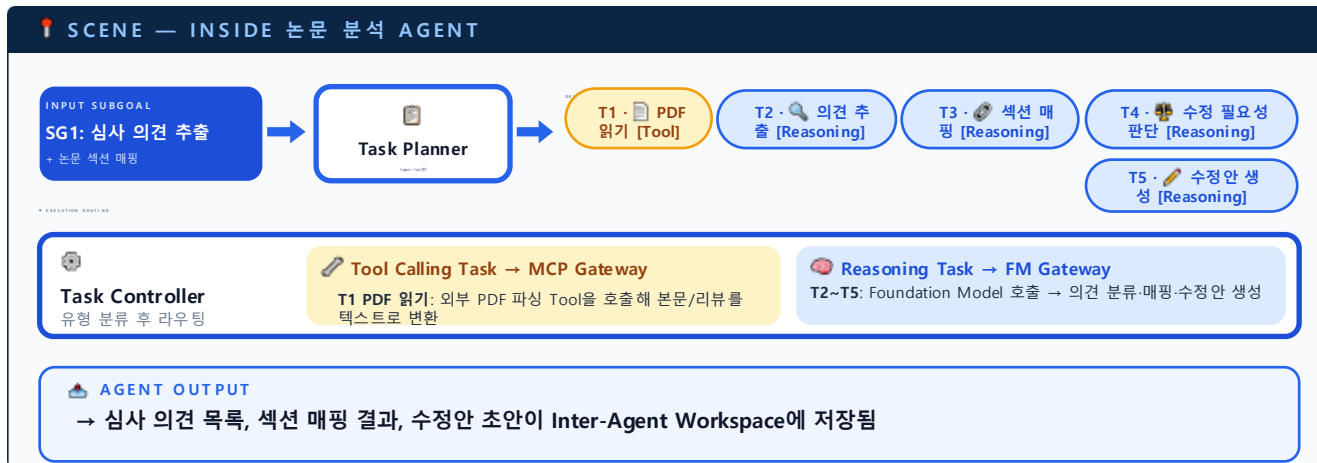
(계획대로 Agent 협업을 실제로 지휘)



STEP
4-5

Task Planner + Task Controller

(Agent 내부에서 Subgoal → Task로 실행)



KEY MESSAGE

“생각하는 일”은 FM Gateway로, “행동하는 일”은 MCP Gateway로 — Task Controller가 자동 분류합니다.

Goal · Subgoal · Task · Workspace의 관계

큰 그릇 안에 작은 그릇이 차곡차곡 들어가는 “계층 구조”입니다.



Q & A
